For You

Presentazione del progetto per Programmazione Mobile

La Rosa Salvatore - a.a. 2024/2025



In questa presentazione:

01 Introduzione

02 User Interface

03 Navigazione

04 Gestione dei Permessi

05 WorkFlow

06 Testing

01

Introduzione



Cos’è ForYou?

L'app For You permette di tracciare e migliorare l’assunzione di farmaci attraverso un'interfaccia semplice e intuitiva.



Requirement

Funzionali

- **Autenticazione locale** (email, PIN, biometria)
- **Gestione farmaci**: aggiunta, modifica, rimozione
- Persistenza locale con **Room Database**
- Liste dinamiche aggiornate in tempo reale (Flow, LiveData)
- **Notifiche** automatiche per i promemoria dei farmaci
- **Resoconti** mensili con grafici e dati aggregati

Non Funzionali

- **Reattività** con Kotlin Coroutines
- Compatibilità da **Android 8.0** (API 26+)
- Scalabilità e manutenibilità tramite **MVVM**

02

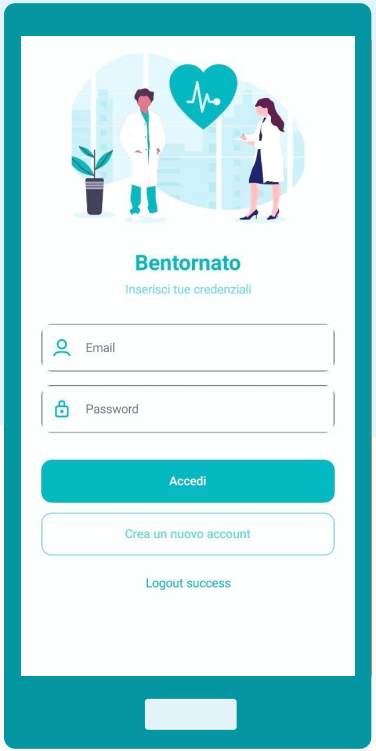
User Interface



Design

Palette utilizza colori tenui definiti secondo lo schema primario, secondario, terziario consigliato dallo standard **MaterialDesign** di Google

È stato realizzato un layout **minimale** per facilitare l'utilizzo e la comprensione



JetPack Compose

Tutta l'interfaccia è una composizione di funzioni *@Composable*.

Abbiamo creato **component** personalizzati e riutilizzabili, come il *ContinuousPinField* o le card per i medicinali, garantendo **coerenza grafica** e manutenibilità.

```
@Composable
fun ContinuousPinField(
    pin: String,
    onPinChange: (String) -> Unit,
    isError: Boolean
) {
    // A list of FocusRequesters to manage focus between the PIN boxes.
    val focusRequesters = List( size = 4) { FocusRequester() }

    Row(...) {
        repeat( times = 4) { index ->
            val char = pin.getOrNull(index)?.toString() ?: ""
            var isFocused by remember { mutableStateOf( value = false) }

            Box(
                modifier = Modifier
                    .size( size = 65.dp)
                    .border(
                        width = 2.dp,
                        color = when {...},
                        shape = MaterialTheme.shapes.medium
                    )
                    .background(...),
                contentAlignment = Alignment.Center
            ) {
                BasicTextField(...)
            }
        }
    }
}
```

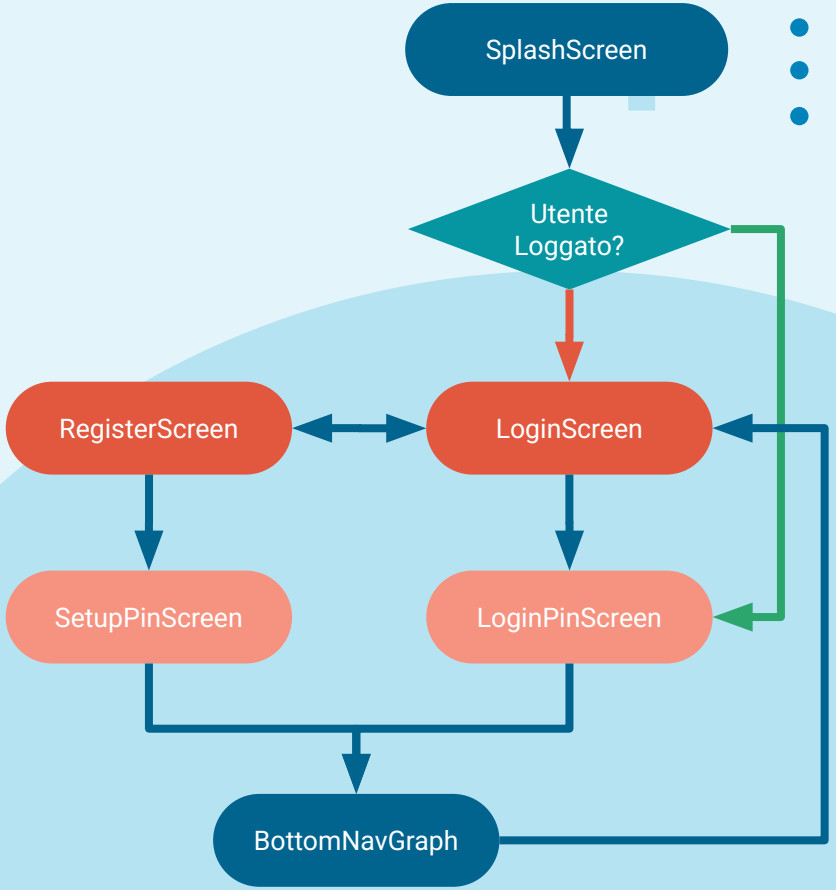
03

Navigazione



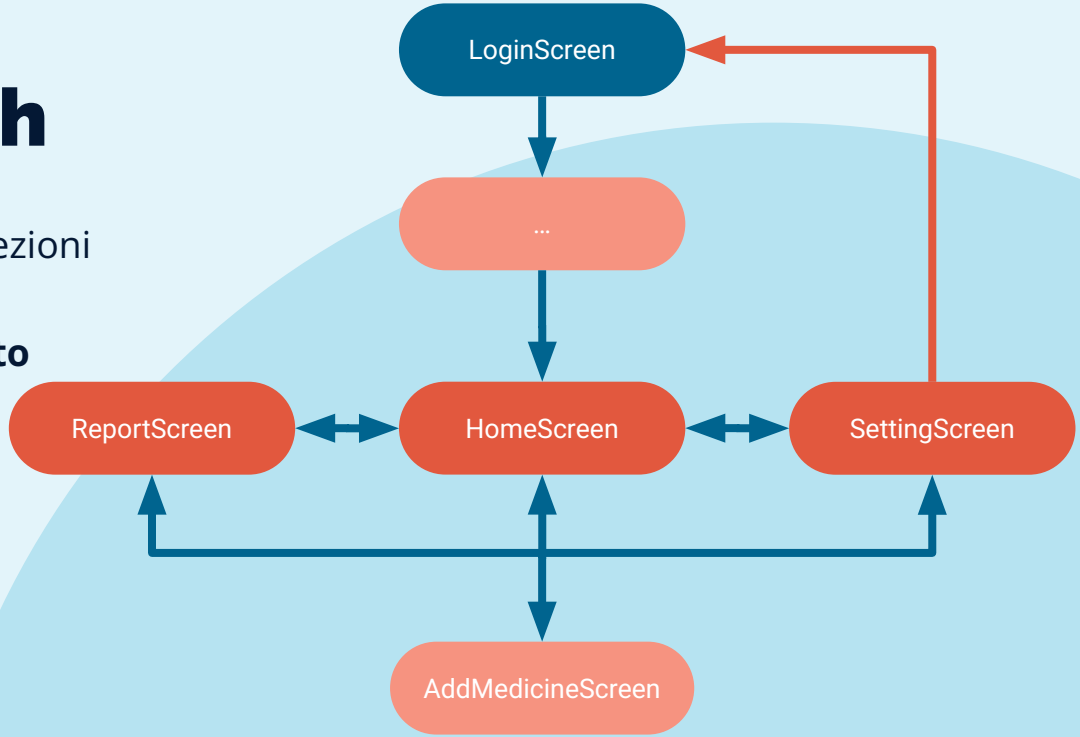
Main NavGraph

- **NavGraph**: definisce destinazioni e azioni globali
- **NavController**: gestisce navigazione e back stack
- **Grafi annidati**: separano flusso iniziale e sezioni della **BottomNavBar**



BottomNavGraph

- **BottomNavBar**: accesso alle sezioni principali **post-login**
- Gestita da un **NavHost dedicato**



04

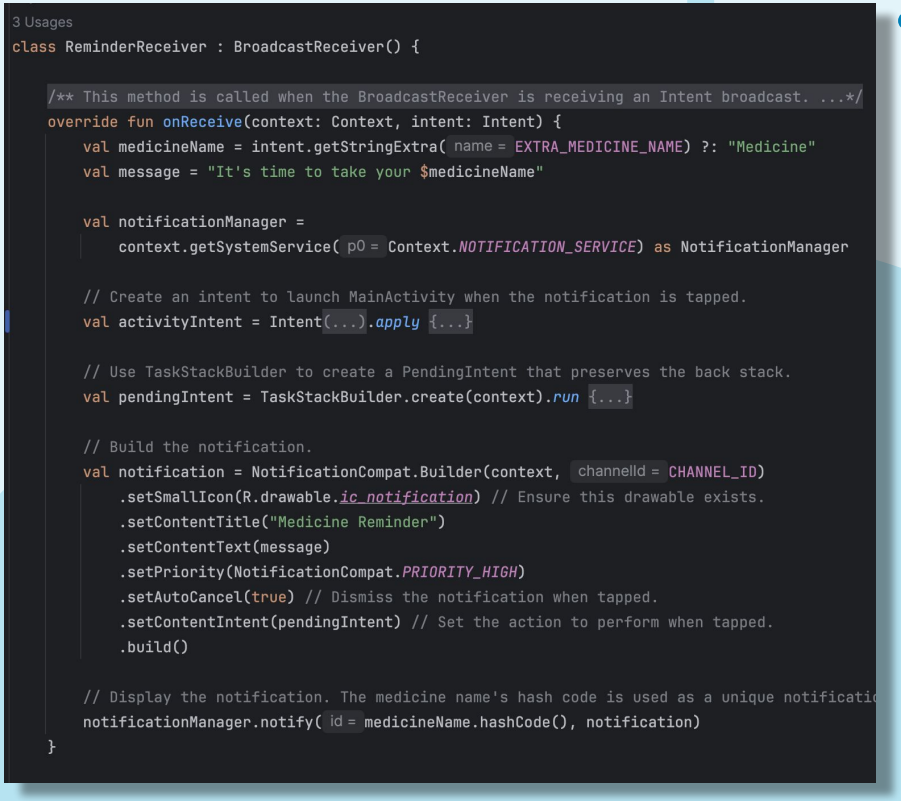
Gestione dei Permessi



Gestione Notifiche

Notifiche push per ricordare l'assunzione dei farmaci:

- *POST_NOTIFICATIONS*: richiesta permesso all'utente
- **AlarmManager**: pianifica le notifiche
- **BroadcastReceiver**: mostra i dettagli del farmaco all'orario previsto



```
3 Usages
class ReminderReceiver : BroadcastReceiver() {

    /** This method is called when the BroadcastReceiver is receiving an Intent broadcast. ...*/
    override fun onReceive(context: Context, intent: Intent) {
        val medicineName = intent.getStringExtra( name = EXTRA_MEDICINE_NAME) ?: "Medicine"
        val message = "It's time to take your $medicineName"

        val notificationManager =
            context.getSystemService( p0 = Context.NOTIFICATION_SERVICE) as NotificationManager

        // Create an intent to launch MainActivity when the notification is tapped.
        val activityIntent = Intent(...).apply {...}

        // Use TaskStackBuilder to create a PendingIntent that preserves the back stack.
        val pendingIntent = TaskStackBuilder.create(context).run {...}

        // Build the notification.
        val notification = NotificationCompat.Builder(context,  channelId = CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification) // Ensure this drawable exists.
            .setContentTitle("Medicine Reminder")
            .setContentText(message)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true) // Dismiss the notification when tapped.
            .setContentIntent(pendingIntent) // Set the action to perform when tapped.
            .build()

        // Display the notification. The medicine name's hash code is used as a unique notificatio
        notificationManager.notify( id = medicineName.hashCode(), notification)
    }
```

```
class BiometricPromptManager(
    1 Usage
    fun showBiometricPrompt(
        title: String,
        description: String
    ) {
        val manager = BiometricManager.from( context = activity)
        val authenticators = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {...} else {
            BIOMETRIC_STRONG
        }

        val promptInfo = BiometricPrompt.PromptInfo.Builder()
            .setTitle(title)
            .setDescription(description)
            .setAllowedAuthenticators(authenticators)
            .apply {
                if (Build.VERSION.SDK_INT < Build.VERSION_CODES.R) {...}
            }
            .build()

        // Controlla la disponibilità del sensore biometrico
        when (manager.canAuthenticate(authenticators)) {
            BiometricManager.BIOMETRIC_ERROR_HW_UNAVAILABLE ->
                resultChannel.trySend( element = BiometricResult.HardwareUnavailable)
            BiometricManager.BIOMETRIC_ERROR_NO_HARDWARE ->
                resultChannel.trySend( element = BiometricResult.FeatureUnavailable)
            BiometricManager.BIOMETRIC_ERROR_NONE_ENROLLED ->
                resultChannel.trySend( element = BiometricResult.AuthenticationNotSet)
            else -> Unit
        }
```

Biometria

Login biometrico con *androidx.biometric*:

- **BiometricManager**: verifica compatibilità
- **BiometricPrompt**: mostra dialog di sistema
- **Callback**:
  - *onAuthenticationSucceeded*: login automatico
  - *onAuthenticationError*: messaggio d'errore all'utente

05

WorkFlow



Entity

User

Memorizza le informazioni dell'utente, incluso il PIN

Medicine

Contiene tutti i dettagli di un farmaco (nome, dosaggio, orario, ecc.)

MedicineIntake

Per tracciare quando un farmaco è stato assunto (timestamp, medicineId, wasTaken)

Il tutto con le rispettive interfacce DAO per specificare le operazioni CRUD per ciascuna entità.

Aggiungere un nuovo farmaco

Interazione utente:

- **UI (Compose):** l'utente compila il form e preme il pulsante "Aggiungi"
- Chiamata a *MedicineViewModel.addMedicine()*

```
// Button to add the medicine.
Button(
    onClick = {
        // Validate fields and insert the medicine.
        if (nome.isNotBlank() && quantita.isNotBlank() && dataFine != null && orario != null) {
            val medicine = Medicine(
                name = nome,
                frequency = frequenza,
                quantity = quantita,
                endDate = dataFine.toString(),
                notes = note.ifBlank { null },
                userEmail = userEmail,
                creationDate = LocalDate.now().toString(),
                time = orario.toString()
            )
            medicineViewModel.insertMedicine(medicine)
            scheduleMedicineReminder(context, nome, orario = orario!!, frequenza)
            navController.popBackStack()
        }
    },
```

Aggiungere un nuovo farmaco

```
fun insertMedicine(medicine: Medicine) {
    viewModelScope.launch( context = dispatcher) {
        repository.insertMedicine(medicine)
    }
}
```

```
open suspend fun insertMedicine(medicine: Medicine) = dao.insertMedicine(medicine)
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertMedicine(medicine: Medicine)
```

Logica applicativa:

- **ViewModel**: avvia coroutine in background
- **Repository**: invoca *insertMedicine()*, ponte con DAO
- **DAO**: scrittura su Room con *Dispatchers.IO*

Aggiungere un nuovo farmaco

Aggiornamento dati

- *getAllMedicinesForUser* tramite *Flow<List<Medicine>>* emette lista aggiornata
- **ViewModel** aggiorna **StateFlow**
- **Compose** si ricompone mostrando il nuovo elemento

```
fun getAllMedicinesForUser(email: String): Flow<List<Medicine>> {
    return repository.getAllMedicinesForUser(email)
}
```

```
// Shows the medicine detail dialog if a medicine is selected.
if (selectedMedicine != null) {
    MedicineDetailDialog(
        medicine = selectedMedicine!!,
        onDismissRequest = { selectedMedicine = null },
        onMarkAsTaken = {
            val today = selectedDate.toString()
            medicineViewModel.markAsTaken(...)
            selectedMedicine = null // Closes the dialog after the action.
        },
        onDeleteFromDate = {...}
    )
}
```

```
1 Usage
fun markAsTaken(medicineName: String, userEmail: String, date: String) {
    viewModelScope.launch( context = dispatcher) {
        val intake = MedicineIntake(
            medicineName = medicineName,
            userEmail = userEmail,
            date = date,
            taken = true
        )
        repository.insertIntake(intake)
    }
}
```

Assunzione farmaco

Azione utente:

- L’utente preme il pulsante “Assunto” sulla card del farmaco
- Chiamata a *homeViewModel.markAsTaken(medicineId)*

Assunzione farmaco

Logica applicativa:

- **ViewModel**: avvia coroutine (*viewModelScope.launch*)
- **Repository**: invoca *insertIntake()*, ponte con DAO

```
open suspend fun insertIntake(intake: MedicineIntake) = dao.insertIntake(intake)
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insertIntake(intake: MedicineIntake)
```

Visualizzazione Report

Recupero dati:

- **UI (ReportScreen)**: per ogni farmaco chiama *medicineViewModel.getIntakesForMedicine()* per recuperare lo storico delle assunzioni

```
// Iterate over each medicine to create an individual report Card.
medicines.forEach { med ->
    // Collect the intake history for the specific medicine.
    val intakes by medicineViewModel
        .getIntakesForMedicine(userEmail, medicineName = med.name)
        .collectAsState(initial = emptyList())
```

Visualizzazione Report

```
fun getIntakesForMedicine(email: String, medicineName: String): Flow<List<MedicineIntake>> {
    return repository.getIntakesForMedicine(email, medicineName)
}
```

```
open fun getIntakesForMedicine(email: String, medicineName: String): Flow<List<MedicineIntake>> =
    dao.getIntakesForMedicine(email, medicineName)
```

```
@Query( value = "SELECT * FROM medicine_intake WHERE userEmail = :email AND medicineName = :medicineName")
fun getIntakesForMedicine(email: String, medicineName: String): Flow<List<MedicineIntake>>
```

Accesso ai dati:

- *reportRepository getAdherenceStats(dateRange)*
- **DAO** (TakenLogDao) calcola dosi previste e assunte

Visualizzazione Report

Aggiornamento dati

- *Ogni nuovo log aggiorna automaticamente i Flow*
- *UI: visualizzazione dei risultati tramite calendario*

```
// Display the medicine card only if it is relevant for the displayed month.
if (isInMonth) {
    ReportCard(
        title = med.name,
        intakes = intakes,
        creationDate = creationDate,
        endDate = endDate,
        monthDays = monthDays,
        today = today
    )
}
```

06

Testing



Instrumented Test

Obiettivo: verificare le operazioni del DAO Room

- Database in-memory con *Room.inMemoryDatabaseBuilder()*
- Contesto Android tramite *ApplicationProvider.getApplicationContext()*
- **@Before**: crea DB temporaneo e inizializza MedicineDao
- **@After**: chiude il DB dopo ogni test

```
@Before
fun createDb() {
    val context = ApplicationProvider.getApplicationContext<Context>()
    db = Room.inMemoryDatabaseBuilder(
        context, klass = UserDatabase::class.java
    )
        // Allowing main thread queries, just for testing.
        .allowMainThreadQueries()
        .build()
    medicineDao = db.medicineDao()
}

/** Closes the database after each test to release resources. ...*/
@After
@Throws( ...exceptionClasses = IOException::class)
fun closeDb() {
    db.close()
}
```

Instrumented Test

```kotlin
@Test
@Throws( ...exceptionClasses = Exception::class)
fun insertAndGetMedicine() = runBlocking {
    val medicine = Medicine(
        id = 1,
        name = "Aspirin",
        frequency = "Daily",
        quantity = "1 pill",
        endDate = "2024-12-31",
        time = "10:00",
        notes = "Take with food",
        userEmail = "test@test.com",
        creationDate = "2024-01-01"
    )
    medicineDao.insertMedicine(medicine)
    val allMedicines = medicineDao.getAllMedicinesForUser( email = "test@test.com").first()
    assertEquals( expected = allMedicines[0], actual = medicine)
}
```

- **Inserisce** Medicine con *insertMedicine()*
- **Recupera** con *getAllMedicinesForUser()*
- **Verifica** dati con *assertEquals()*

Unit Test

Obiettivo: testare logica del ViewModel e interazioni col Repository

- **Mockito** per mock del *MedicineRepository*
- *StandardTestDispatcher* per controllare le coroutine
- **@Before / @After**: sostituisce e ripristina *Dispatchers.Main*

```
@Before
fun setup() {
    Dispatchers.setMain(testDispatcher)
    repository = mock()
    viewModel = MedicineViewModel(repository, testDispatcher)
}

/** Cleans up the test environment after each test. ...*/
@After
fun tearDown() {
    Dispatchers.resetMain()
}
```

```
@Test
fun `insertMedicine should call repository's insertMedicine`() = runTest {
    val medicine = Medicine(
        name = "Aspirin",
        frequency = "Daily",
        quantity = "1 pill",
        endDate = "2024-12-31",
        time = "10:00",
        notes = "Take with food",
        userEmail = "test@test.com",
        creationDate = "2024-01-01")
    viewModel.insertMedicine(medicine)
    // Advances the scheduler to execute the coroutine launched by the ViewModel.
    testDispatcher.scheduler.advanceUntilIdle()
    // Verifies that the suspend function on the repository was called.
    verify( mock = repository).insertMedicine(medicine)
}
```

Unit Test

Test principali:

- *insertMedicine()*: verifica chiamata al repository
- *getAllMedicinesForUser()*: controlla dati restituiti
- *getIntakesForMedicine()*: verifica propagazione dei Flow

Test sul campo

- **Partecipanti**: 4 utenti (25–85 anni), 5 dispositivi di fasce diverse
- **Scenario di test**: Registrazione → Aggiunta farmaco → Notifica → Conferma assunzione
- **Obiettivo**: valutare usabilità, affidabilità e compatibilità
- **Risultati generali**: App stabile su tutti i dispositivi
- **Autenticazione biometrica**: efficace salvo limiti hardware
- **Notifiche**: puntuali, salvo errori di permessi utente



Osservazioni per tipologia di utente

Esperto (25 anni)

- esperienza fluida
- solo piccole scomodità nei text field

Semi esperta (25 anni)

- app intuitiva
- input da tastiera poco pratico

Casual (53–59 anni)

- Buona usabilità generale
- Problemi minori con biometria o permessi notifiche
- Preferire palette con maggior contrasto

Anziano (85 anni)

- Difficoltà di interazione autonoma
- Testo piccolo, contrasto basso, problemi di percezione tap
- Necessario migliorare accessibilità (dimensioni, colori, feedback visivi)

Grazie!

La Rosa Salvatore
For You